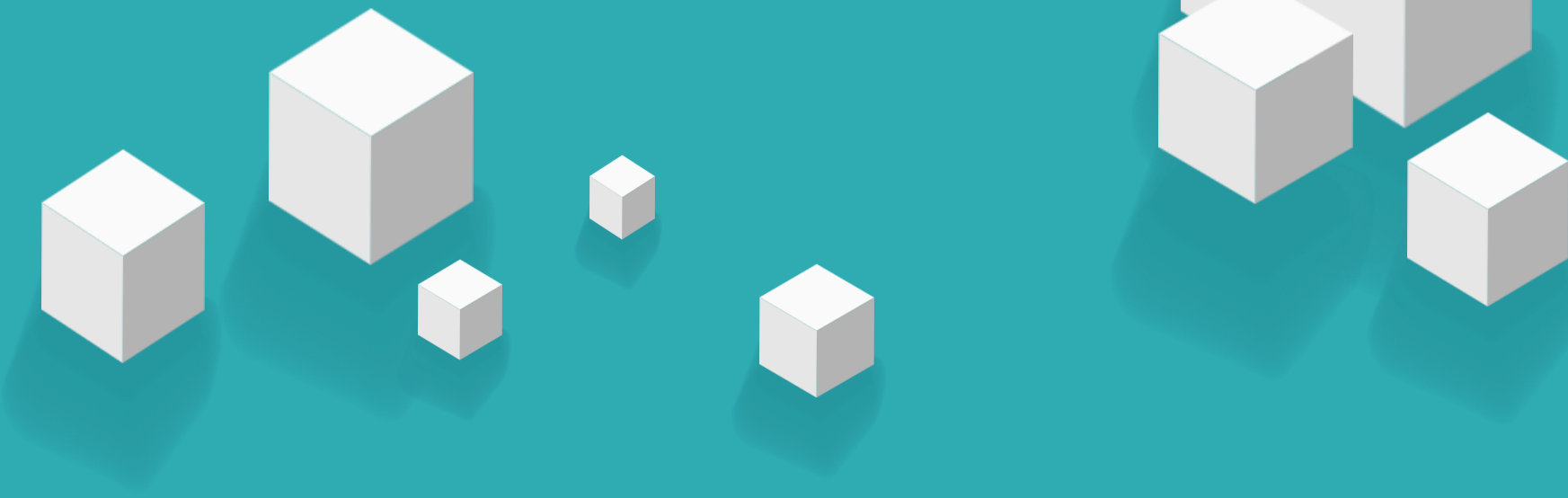


vue.js
Aug 12

第八章 组件基础（上）





8.1

选项式API和组合式API

8.2

生命周期函数

8.3

组件的注册和引用

8.4

组件之间的数据传递



8.1

选项式API和组合式API

8.1 选项式API和组合式API

Vue 3支持选项式API和组合式API。其中，选项式API是从Vue 2开始使用的一种写法，而Vue 3新增了组合式API的写法。



选项式API

选项式API是一种通过包含多个选项的对象来描述组件逻辑的API，其常用的选项包括data、methods、computed、watch等。

组合式API

相比于选项式API，组合式API是将组件中的数据、方法、计算属性、侦听器等代码全部组合在一起，写在setup()函数中。

8.1 选项式API和组合式API

■ 选项式API的语法格式。

```
<script>
export default {
  data() {
    return { // 定义数据 }
  },
  methods: { // 定义方法 },
  computed: { // 定义计算属性 },
  watch: { // 定义侦听器 }
}
</script>
```

■ 组合式API的语法格式。

```
<script>
import { computed, watch } from 'vue'
export default {
  setup() {
    const 数据名 = 数据值
    const 方法名 = () => {}
    const 计算属性名 = computed(() => {})
    watch(侦听器的来源, 回调函数, 可选参数)
    return { 数据名, 方法名, 计算属性名 }
  }
}
</script>
```

8.1 选项式API和组合式API

Vue使用`setup`语法糖时，组合式API的语法格式如下。

```
<script>
import { computed, watch } from 'vue'
export default {
  setup() {
    const 数据名 = 数据值
    const 方法名 = () => {}
    const 计算属性名 = computed(() => {})
    watch(侦听器的来源, 回调函数, 可选参数)
    return { 数据名, 方法名, 计算属性名 }
  }
}
</script>
```

```
<script setup>
import { computed, watch } from 'vue'
const 数据名 = 数据值
const 方法名 = () => {}
const 计算属性名 = computed(() => {})
watch(侦听器的来源, 回调函数, 可选参数)
</script>
```

setup语法糖



<script>

```
var vm = new Vue({
  el: '#app',
  data: {
    list: [
      { productId: 1, name: '柯西' },
      { productId: 2, name: '泊松' },
      { productId: 3, name: '余弦' },
      { productId: 4, name: '正弦' }
    ]
  },
  ...
})
```

Vue2

<script setup>

```
import { ref, computed } from 'vue'
```

```
const list = ref([
  { productId: 1, name: '柯西' },
  { productId: 2, name: '泊松' },
  { productId: 3, name: '余弦' },
  { productId: 4, name: '正弦' }
])
```

```
const total = computed(() => {
  return list.value.reduce((sum, item) => {
    return item.selected ? sum + item.productId : sum
  }, 0)
})
```

</script>

<script>

```
export default {
  name: 'goods',
  data() {
    return {
      list: [
        { productId: 1, name: '柯西' },
        { productId: 2, name: '泊松' },
        { productId: 3, name: '余弦' },
        { productId: 4, name: '正弦' }
      ]
    }
  },
  ...
}
```

Vue3 选项API

> computed: { ... }

</script>

```
import { createApp } from 'vue'
const app = createApp({
  // ...
}).mount('#app')
```

Vue3 组合API
语法糖

< index.html

App.vue

JS main.js



hello-vite > src > JS main.js

```
1 import { createApp } from 'vue'
2 import './style.css'
3 import App from './App.vue'
4
5 createApp(App).mount('#app')
6
```

8.1 选项式API和组合式API

选项式API和组合式API的关系

Vue提供的选项式API和组合式API这两种写法可以覆盖大部分的应用场景，它们是同一底层系统所提供的两套不同的接口。选项式API是在组合式API的基础上实现的。

大型项目开发时，随着业务复杂度的增加，代码量会不断增加。

选项式API

- 整个项目逻辑不易阅读和理解
- 查找对应功能的代码存在一定难度。

组合式API

- 将项目的每个功能的数据、方法放到一起，可以快速定位到功能区域相关代码，便于阅读和维护。
- 通过函数来实现高效的逻辑复用，形式更加自由，需要开发者有较强的代码组织能力和拆分逻辑能力。

8.1 选项式API和组合式API

■ 选项式API

src > components > OptionsAPI.vue > ...

```
1 <template>
2   <div>数字: {{ number }}</div>
3   <button @click="addNumber">+1</button>
4 </template>
5
6 <script>
7 export default {
8   data() {
9     return {
10       number: 1
11     }
12   },
13   methods: {
14     addNumber() {
15       this.number++
16     }
17   }
18 }
19 </script>
```

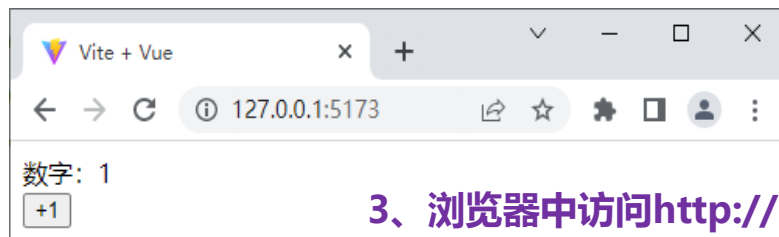
■ 组合式API

src > components > CompositionAPI.vue > ...

```
1 <template>
2   <div>数字: {{ number }}</div>
3   <button @click="addNumber">+1</button>
4 </template>
5
6 <script setup>
7   import { ref } from 'vue'
8   let number = ref(1)
9   const addNumber = () => {
10     number.value ++
11   }
12 </script>
```

2、修改src/main.js文件。

```
import App from './components/OptionsAPI.vue'
```



3、浏览器中访问http://127.0.0.1:5173/

8.2 生命周期函数

- 在Vue中，组件的**生命周期**是指**每个组件从被创建到被销毁的整个过程**，每个组件都有生命周期。
- 想要**在某个特定的时机进行特定的处理**，可以通过**生命周期函数**来完成。
- 随着组件**生命周期的变化**，生命周期函数会**自动执行**。

8.2 生命周期函数

组合式API下的生命周期函数如下表所示。

函数	说明
onBeforeMount()	组件挂载前
onMounted()	组件挂载成功后
onBeforeUpdate()	组件更新前
onUpdated()	组件中的任意的DOM元素更新后
onBeforeUnmount()	组件实例被销毁前
onUnmounted()	组件实例被销毁后

8.2 生命周期函数

以`onMounted()`函数为例演示生命周期函数的使用。

```
<script setup>
import { onMounted } from 'vue'
onMounted(() => {
  // 执行操作
})
</script>
```

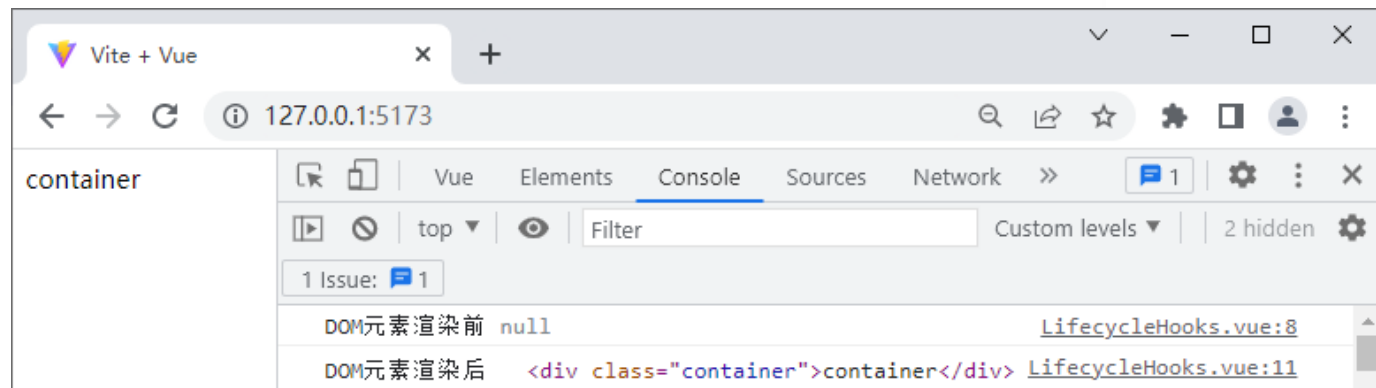
8.2 生命周期函数

生命周期函数的使用方法

创建src\components\LifecycleHooks.vue文件，用于通过生命周期函数查看在特定时间点下的DOM元素。

src > components > LifecycleHooks.vue > ...

```
1 <template>
2   <div class="container">container</div>
3 </template>
4 <script setup>
5   import { onBeforeMount, onMounted } from 'vue'
6   onBeforeMount(() => {
7     console.log('DOM元素渲染前', document.querySelector('.container'))
8   })
9   onMounted(() => {
10    console.log('DOM元素渲染后', document.querySelector('.container'))
11  })
12 </script>
```



8.2 生命周期函数

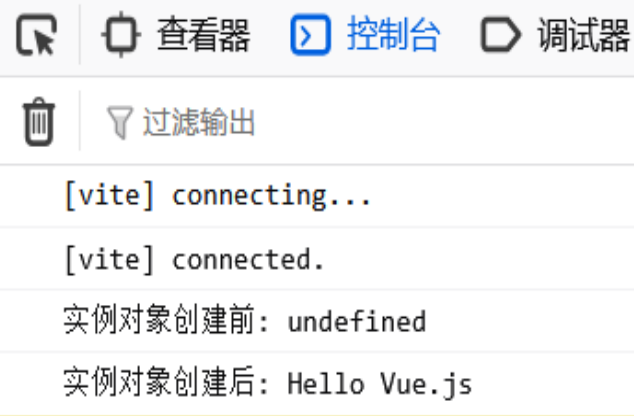
选项式API下的生命周期函数。

函数	说明	
beforeCreate()	实例对象创建前	
created()	实例对象创建后	组合API 函数
beforeMount()	页面挂载前	<u>onBeforeMount()</u>
mounted()	页面挂载成功后	<u>onMounted()</u>
beforeUpdate()	组件更新前	<u>onBeforeUpdate()</u>
updated()	组件中的任意的DOM元素更新后	<u>onUpdated()</u>
beforeDestroy()	组件实例销毁前	<u>onBeforeUnmount()</u>
destroyed()	组件实例销毁后	onUnmounted()

8.2 生命周期函数

选项式API下beforeCreate()函数和created()函数的使用。

```
<script>
export default {
  data() {
    return {
      value: 'Hello Vue.js '
    }
  },
  beforeCreate() {
    console.log('实例对象创建前: ' + this.value)
  },
  created() {
    console.log('实例对象创建后: ' + this.value)
  }
}
</script>
```



8.3 组件的注册和引用

8.3.1 注册组件

- 在Vue项目中定义了一个新的组件后，要想在其他组件中引用这个新的组件，需要对新的组件进行注册。
- 注册组件时，需要给组件取名字从而区分每个组件，可以采用帕斯卡命名法（PascalCase）为组件命名。
- Vue提供了两种注册组件的方式
 - 全局注册
 - 局部注册。

PascalCase规则：每个单词的首字母大写，单词之间无分隔符（如下划线或空格）

8.3 组件的注册和引用

8.3.1 注册组件

1. 全局注册

- **全局注册**：在实际开发中，某个组件的**使用频率很高**，许多组件中都会引用该组件，则推荐将该组件注册为全局组件。
- **被全局注册的组件可以在当前Vue项目的任何一个组件内引用。**
- 在src\main.js文件中，通过Vue应用实例的**component()**方法可以**全局注册组件**，语法格式如下。

component('组件名称', 需要被注册的组件)

其中，**component()**方法接收**两个参数**，

- ◆ **组件名称**，注册完成后即可全局使用该组件名称。
- ◆ **需要被注册的组件**。

8.3 组件的注册和引用

8.3.1 注册组件

在src\main.js文件中注册一个全局组件MyComponent，例如。

```
import { createApp } from 'vue';  
import './style.css'  
import App from './App.vue'  
import MyComponent from './components/MyComponent.vue'  
const app = createApp(App)  
app.component('MyComponent', MyComponent)  
app.mount('#app')
```

component()方法支持链式调用，可以连续注册多个组件，例如。

```
app.component('ComponentA', ComponentA)  
  .component('ComponentB', ComponentB)  
  .component('ComponentC', ComponentC)
```

8.3 组件的注册和引用

8.3.1 注册组件

2. 局部注册

- 局部注册：某些组件只在特定的情况下被用到，推荐进行局部注册。
- 局部注册即在某个组件中注册，该组件只能在当前注册范围内使用。例如。

```
<script>
import ComponentA from './ComponentA.vue'
export default {
  components: { ComponentA: ComponentA }
}
</script>
```

使用`setup`语法糖，导入的组件会被自动注册，导入后可以直接在模板中使用。

```
<script setup>
import ComponentA from './ComponentA.vue'
</script>
```

8.3 组件的注册和引用

8.3.2 引用组件

将组件注册完成后，若要将组件在页面中渲染出来，需要引用组件。

- 在组件的 `<template>` 标签中可以引用其他组件，被引用的组件需要写成标签的形式，标签名应与组件名对应。
- 组件的标签名可以使用短横线分隔或帕斯卡命名法命名。
 - `<my-component>` 标签
 - `<MyComponent>` 标签
- 一个组件可以被引用多次，但不可出现自我引用和互相引用的情况，否则会出现死循环。

回顾1 选项式API和组合式API

Vue 3支持选项式API (Vue2) 和组合式API(Vue3)。



选项式API

- 通过包含多个选项的对象来描述组件逻辑的API
- 常用的选项包括data、methods、computed、watch等。

组合式API

- 将组件中的数据、方法、计算属性、侦听器等代码全部组合在一起，写在setup()函数中。

回顾2 生命周期函数

选项API函数	组合API函数	说明
beforeCreate()		实例对象创建前
created()		实例对象创建后
beforeMount()	onBeforeMount()	页面/组件挂载前
mounted()	onMounted()	页面/组件挂载成功后
beforeUpdate()	onBeforeUpdate()	组件更新前
updated()	onUpdated()	组件中的任意的DOM元素更新后
beforeDestroy()	onBeforeUnmount()	组件实例销毁前
destroyed()	onUnmounted()	组件实例销毁后

>>> 回顾3 组件的注册和引用

1、注册组件

```
import { createApp } from 'vue';
```

■ 全局组件

```
import './style.css'
```

```
import App from './App.vue'
```

```
import MyComponent from './components/MyComponent.vue'
```

```
const app = createApp(App)
```

```
app.component('MyComponent', MyComponent)
```

```
app.mount('#app')
```

组件名

组件

```
<script setup>
```

```
import ComponentA from './ComponentA.vue'
```

```
</script>
```

■ 局部组件

2、引用组件

■ <my-component> 标签

短线分隔

■ <MyComponent> 标签

pascal命名

```
<script>
```

```
import ComponentA from './ComponentA.vue'
```

```
export default {
```

```
  components: { ComponentA: ComponentA }
```

```
}
```

简写 **components: { ComponentA }**

```
</script>
```

8.3 组件的注册和引用

※组件的使用方法示例※

```
<template>
  <h5>父组件</h5>
  <div class="box"></div>
</template>
```

```
<style>
.box {
  display: flex;
}
</style>
```

1

①创建文件src\components\ComponentUse.vue

②全局组件：src\components\GlobalComponent.vue。

③局部组件：src\components\LocalComponent.vue。

```
<template>
  <div class="global-container">
    <h5>全局组件</h5>
  </div>
</template>
<style>
.global-container {
  border: 1px solid black;
  height: 50px;
  flex: 1;
}
</style>
```

2

```
<template>
  <div class="local-container">
    <h5>局部组件</h5>
  </div>
</template>
<style>
.local-container {
  border: 1px dashed black;
  height: 50px;
  flex: 1;
}
</style>
```

3

8.3 组件的注册和引用

※组件的使用方法示例※

④修改src\main.js文件，导入GlobalComponent组件并调用component()方法
全局注册GlobalComponent组件。

```
import { createApp } from 'vue'
import './style.css'
import App from './components/ComponentUse.vue'
import GlobalComponent from './components/GlobalComponent.vue'
const app = createApp(App)
app.component('GlobalComponent', GlobalComponent)
app.mount('#app')
```

⑤修改src\components\ComponentUse.vue文件，导入LocalComponent组件。

```
<script setup>
import LocalComponent from './LocalComponent.vue'
</script>
```

8.3 组件的注册和引用

※组件的使用方法示例※

⑥修改src\components\ComponentUse.vue文件，引用GlobalComponent组件和LocalComponent组件。

```
<div class="box">  
  <GlobalComponent />  
  <LocalComponent />  
</div>
```

注意

<GlobalComponent />(自闭合标签)

- 用途：当组件不需要包裹任何子元素或内容时使用。
- 特点：
 - 直接闭合，没有内部内容。
 - 等同于 <GlobalComponent> </GlobalComponent>（空内容）。

8.3 组件的注册和引用

※组件的使用方法示例※

⑦修改后的src\components\ComponentUse.vue文件。

```
1  <template>
2    <h5>父组件</h5>
3    <div class="box">
4      <GlobalComponent />
5      <LocalComponent />
6    </div>
7  </template>
8
9  <script setup>
10    import LocalComponent from './LocalComponent.vue'
11  </script>
12
13  <style>
14    .box {
15      display: flex;
16    }
17  </style>
```

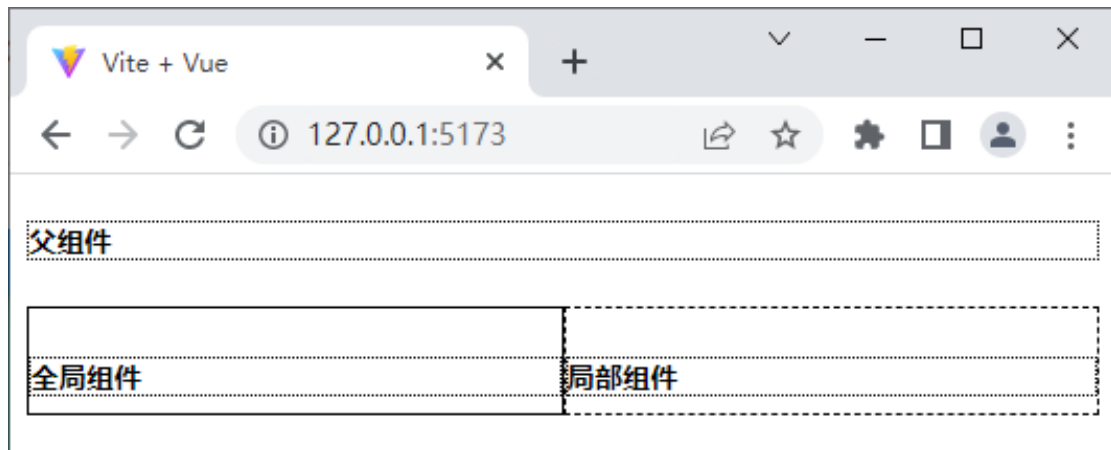


8.3 组件的注册和引用

8.3.3 解决组件之间的样式冲突

默认写在Vue组件中的样式会全局生效，很容易造成多个组件之间的样式冲突问题。例如，为ComponentUse组件中的h5元素添加边框样式。

```
h5 {  
  border: 1px dotted black;  
}
```



ComponentUse组件、GlobalComponent组件和LocalComponent组件中h5元素的边框样式都发生了改变，而代码中只有ComponentUse组件设置了边框样式效果，说明组件之间存在样式冲突。

8.3 组件的注册和引用

8.3.3 解决组件之间的样式冲突

- **根本原因**：在单页Web应用中，所有组件的DOM结构都是基于唯一的index.html页面进行呈现的。
- 每个组件中的样式都可以影响整个页面中的DOM元素。
- **解决方案**：
 - scoped属性
 - 深度选择器

8.3 组件的注册和引用

8.3.3 解决组件之间的样式冲突

1. scoped属性

- Vue为<style>标签提供了scoped属性，用于解决组件之间的样式冲突。
- 为<style>标签添加scoped属性后，Vue会自动为当前组件的DOM元素添加一个唯一的自定义属性（如data-v-7ba5bd90），并在样式中为选择器添加自定义属性（如.list[data-v-7ba5bd90]），从而限制样式的作用范围，防止组件之间的样式冲突问题。

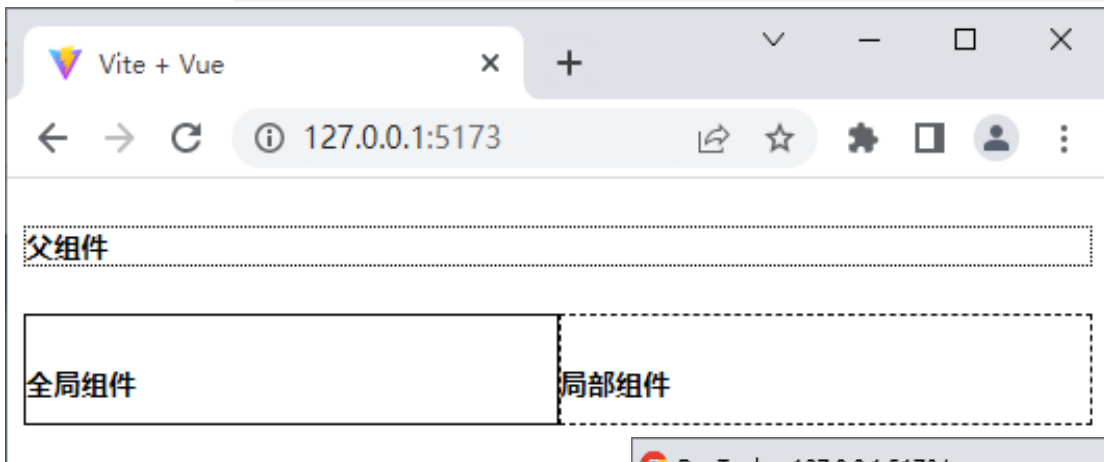
8.3 组件的注册和引用

示例——scoped属性解决样式冲突

8.3.3 解决组件之间的样式冲突

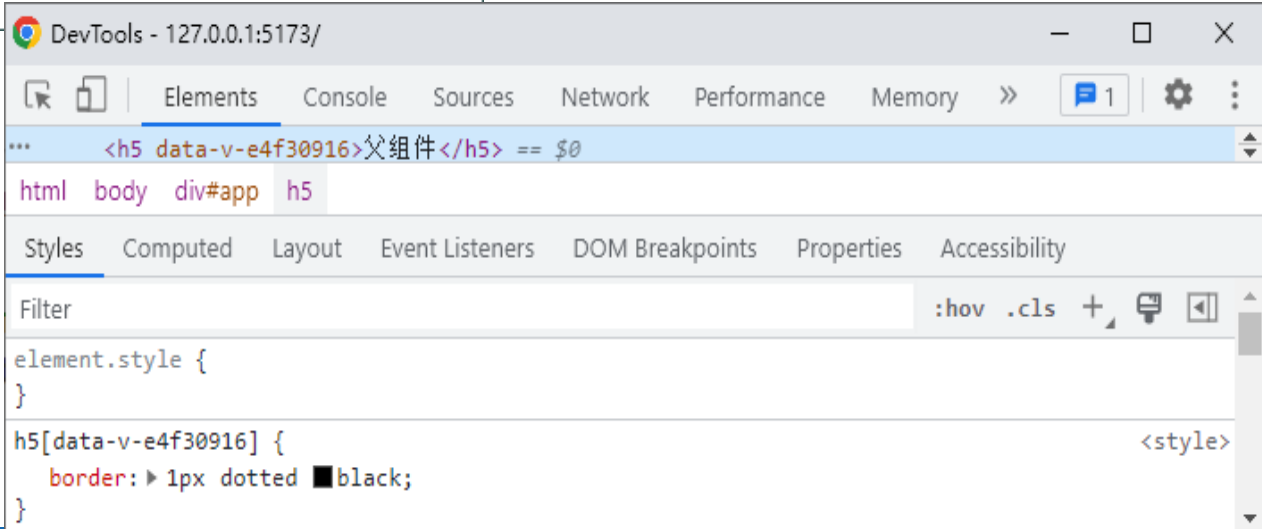
示例：修改ComponentUse组件，为<style>标签添加scoped属性。

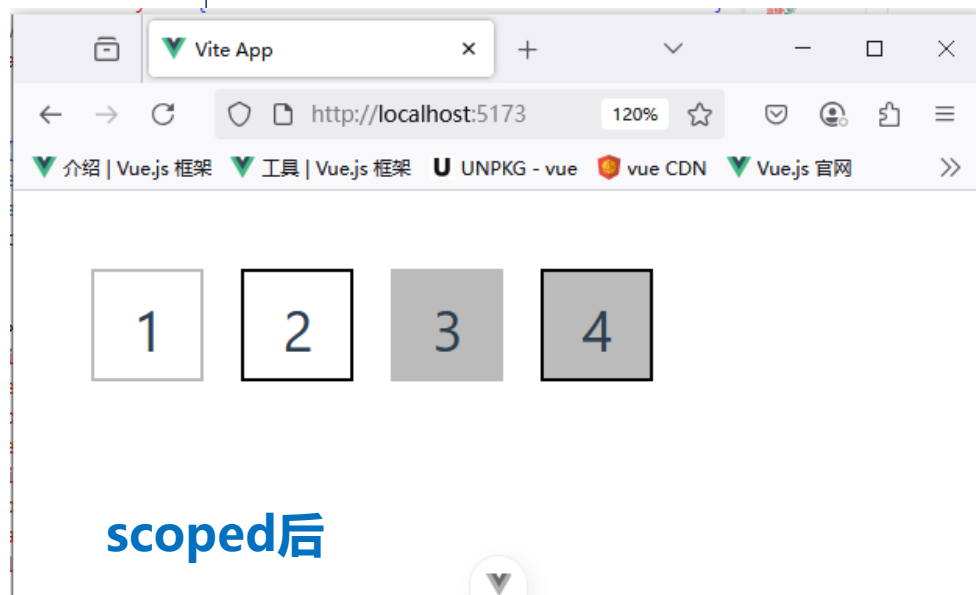
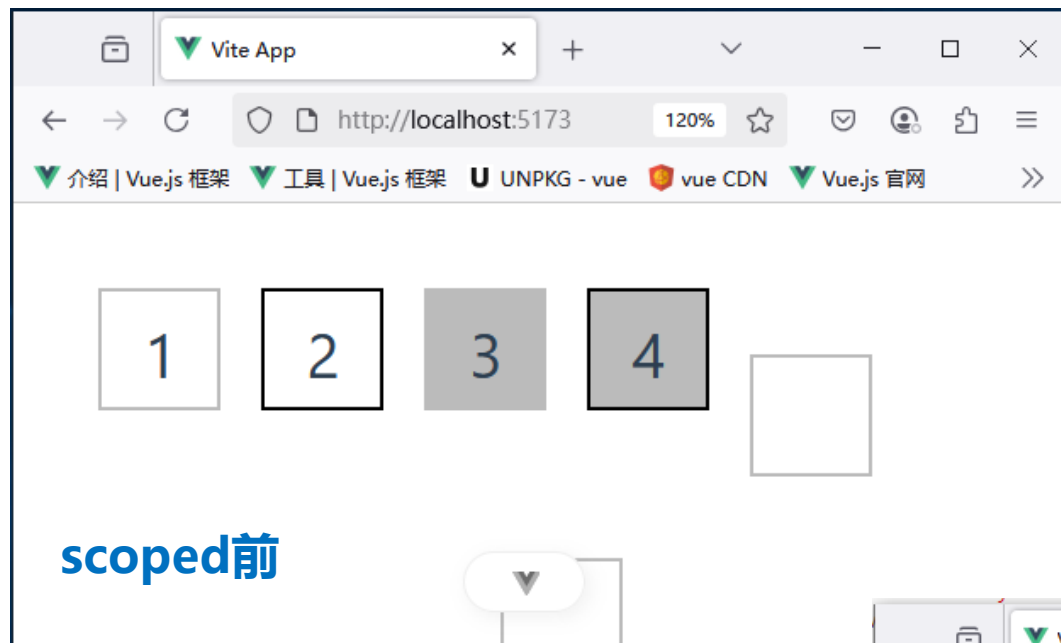
<style scoped>



使用开发者工具切换到Elements面板，查看父组件的h5元素的代码，h5元素和相应的选择器被Vue自动添加了data-v-e4f30916属性，从而解决了样式冲突问题。

修改后，在浏览器中访问
<http://127.0.0.1:5173/>





8.3 组件的注册和引用

8.3.3 解决组件之间的样式冲突

2. 深度选择器

- 当前组件的<style>标签添加了scoped属性，则该组件的样式对其子组件不生效。如果在添加了scoped属性后还需要让某些样式对子组件生效，则可以使用深度选择器来实现。
- 深度选择器通过:deep()伪类来实现，在其小括号中可以定义用于子组件的选择器，例如，“:deep(.title)”被编译之后生成选择器的格式为 “[data-v-7ba5bd90] .title”。

>> 8.3 组件的注册和引用

示例——深度选择器解决样式冲突

❶ 为LocalComponent组件的h5元素添加class属性。

```
<template>
  <div class="local-container">
    <h5 class="title"> >局部组件</h5>
  </div>
</template>
<style> </style>
```

❷ 在ComponentUse组件中定义.title的样式。

```
<template>
  <h5>父组件</h5>
  <div class="box"> </div>
</template>
<style>
  :deep(.title){
    border: 3px dotted black;
  }
</style>
```

❸ 在浏览器中访问http://127.0.0.1:5173/



8.4 组件间的数据传递

8.4.1 父组件向子组件传递

1 声明props

- 若想实现父组件向子组件传递数据，需要先在子组件中声明props，表示子组件可以从父组件中接收哪些数据。
- 不使用setup语法糖的情况下，可以使用props选项声明props。props选项的形式可以是对象或字符串数组。
 - 声明对象形式的props的语法。
 - 声明字符串数组形式的props语法。

```
<script>
export default {
  props: {
    自定义属性A: 类型,
    自定义属性B: 类型,
    .....
  }
}
</script>
```

(如果不需要限制props的类型)

```
props: ['自定义属性A', '自定义属性B'],
```

8.4.1 父组件向子组件传递

1 声明props

- 使用setup语法糖，使用defineProps()函数。

- 声明对象形式props。

```
<script setup>  
const props = defineProps({'自定义属性A': 类型}, {'自定义属性B': 类型})  
</script>
```

- 声明字符串数组形式的props。

```
const props = defineProps(['自定义属性A', '自定义属性B'])
```

- 在组件中声明了props后，可以直接在模板中输出每个prop的值。

```
<template>  
  {{ 自定义属性A }}  
  {{ 自定义属性B }}  
</template>
```

8.4.1 父组件向子组件传递

2 静态绑定props

当在父组件中引用了子组件后，如果子组件中声明了props，则可以在父组件中向子组件传递数据。

- 如果传递的数据是固定不变的，则可以通过静态绑定props的方式为子组件传递数据。
- 通过静态绑定props的方式为子组件传递数据，其语法格式如下。

```
<子组件标签名 自定义属性A="数据" 自定义属性B="数据" />
```

说明：父组件向子组件的props传递了静态的数据，属性值默认为字符串类型。

注意

如果子组件中未声明props，则父组件向子组件中传递的数据会被忽略，无法被子组件使用。

8.4.1 父组件向子组件传递

※静态绑定props示例※

① 创建src\components\Count.vue文件——子组件。

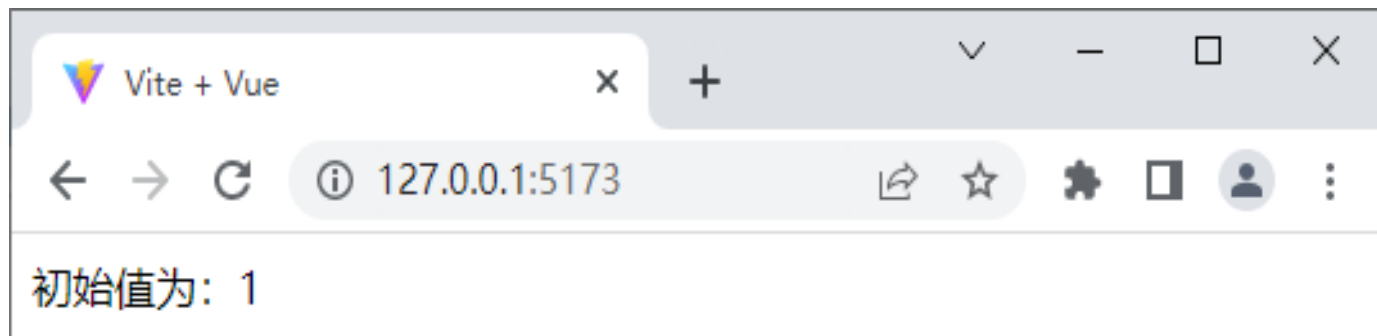
```
1 <template>
2   初始值为: {{ num }}
3 </template>
4 <script setup>
5   const props = defineProps({
6     num: String
7   })
8 </script>
```

② 创建src\components\Props.vue文件——父组件。

```
1 <template>
2   <Count num="1" />
3 </template>
4 <script setup>
5   import Count from './Count.vue'
6 </script>
```

③ 修改src\main.js文件。 import App from './components/Props.vue'

④ 在浏览器中访问http://127.0.0.1:5173/



»» 示例——props属性

Vite App

▼ student.vue ×

▼ App.vue

src > components > ▼ student.vue > {} script

```
1 <template>
2   <div>
3     <h2>学生姓名: {{ name }}</h2>
4     <h2>学生性别: {{ sex }}</h2>
5     <h2>学生年龄: {{ age }}</h2>
6   </div>
7 </template>
8
9 <script>
10
11   export default{
12     name:'student',
13     data(){
14       return{
15         msg:'我是一个工大的学生',
16       }
17     },
18     props:['name','sex','age']
19   }
20 </script>
```

Vite App

▼ student.vue

▼ App.vue ×

src > ▼ App.vue > ...

```
1 <script setup>
2   import Student from '@components/student.vue';
3
4 </script>
5
6 <template>
7   <div>
8     <Student name="李四" sex="女" age="20"/>
9   </div>
10 </template>
```

Vite App ×

▼ student.vue

← → ↺ http://localhost:4000/

学生姓名: 李四
学生性别: 女
学生年龄: 20

8.4.1 父组件向子组件传递

3、动态绑定props

在父组件中使用v-bind可以为子组件动态绑定props，任意类型的值都可以传给子组件的props，包括字符串、数字、布尔值、数组、对象等。

8.4.1 父组件向子组件传递

3、 动态绑定props

1. 字符串

- 从父组件中为子组件传递字符串类型的props数据。

```
1  <template>
2  |   <Child :init="username" />
3  </template>
4  <script setup>
5  import Child from './Child.vue'
6  import { ref } from 'vue'
7  const username = ref('小圆')
8  </script>
```

init属性是子组件中声明的props。

- 名称为Child的子组件代码。

```
1  <template></template>
2  <script setup>
3  const props = defineProps(['init'])
4  console.log('子组件props:', props)
5  </script>
```

props.init

查看器 控制台 调试器 网络 样式



过滤输出

错误

警告

[vite] connecting...

[vite] connected.

子组件props: ▶ Proxy { <target>: Proxy, <handler>: {...} }

8.4.1 父组件向子组件传递

3、动态绑定props

2. 数字

从父组件中为子组件传递数字类型的props数据，示例代码。

```
1 <template>
2   <Child :init="12" /> 12为表达式
3   <Child :init="age" /> 动态赋值
4 </template>
5 <script setup>
6 import Child from './Child.vue'
7 import { ref } from 'vue'
8 const age = ref(13) 数字类型
9 </script>
```



8.4.1 父组件向子组件传递

3、动态绑定props

3. 布尔值

从父组件中为子组件传递布尔类型的props数据，示例代码。

```
1  <template>
2    <Child init /> init设为true
3    <Child :init="false" /> 表达式
4    <Child :init="isFlag" /> 动态赋值
5  </template>
6  <script setup>
7    import Child from './Child.vue'
8    import { ref } from 'vue'
9    const isFlag = ref(true)
10 </script>
```

[vite] connecting...

[vite] connected.

► Proxy { <target>: Proxy, <handler>: {...} }

<empty string>

► Proxy { <target>: Proxy, <handler>: {...} }

false

► Proxy { <target>: Proxy, <handler>: {...} }

true

未声明

```
const props = defineProps({
  init: {
    type: Boolean,
    default: false
  }
});
```

8.4.1 父组件向子组件传递

3、动态绑定props

4. 数组

从父组件中为子组件传递数组类型的props数据，示例代码。

```
1  <template>
2    <Child :init="['唱歌', '跳舞', '滑冰']" />  表达式-数组
3    <Child :init="hobby" />    动态赋值
4  </template>
5  <script setup>
6    import Child from './Child.vue'
7    import { ref } from 'vue'
8    const hobby = ref(['唱歌', '跳舞', '滑冰'])
9  </script>
```

[vite] connecting...

[vite] connected.

▶ Array(3) ["唱歌", "跳舞", "滑冰"]

▼ Proxy { <target>: (3) [...], <handler>: {...} }

▶ <target>: Array(3) ["唱歌", "跳舞", "滑冰"]

▶ <handler>: Object { _isReadonly: false, _isShallow: false }

8.4.1 父组件向子组件传递

3、动态绑定props

5. 对象

从父组件中为子组件传入一个对象类型的props数据，或者将对象中的部分属性作为被传入的props数据，示例代码如下。

```
1  <template>
2    <Child :init="{ height: '180厘米', weight: '70千克' }" />
3    <Child :height="bodyInfo.height" :weight="bodyInfo.weight" />
4    <Child v-bind="bodyInfo" /> //将对象的所有属性传入，简化3行
5  </template>
6  <script setup>
7    import Child from './Child.vue'
8    import { reactive } from 'vue'
9    const bodyInfo = reactive({
10      height: '180厘米',
11      weight: '70千克'
12    })
13  </script>
```

```
console.log(props.init)
console.log(props.weight)
```

```
[vite] connecting...
[vite] connected.
▶ Object { height: "180厘米", weight: "70千克" }
undefined
undefined
70千克
undefined
70千克
```

8.4.1 父组件向子组件传递



props单向数据流

- 在Vue中，所有的props都遵循单向数据流原则，props数据因父组件的更新而变化，变化后的数据将向下流往子组件，而且不会逆向传递，这样可以防止因子组件意外变更props导致数据流向难以理解的问题。
- 每次父组件绑定的props发生变更时，子组件中的props都将会刷新为最新的值。
- 开发者不应在子组件内部改变props，否则Vue会在浏览器的控制台中警告。

8.4.1 父组件向子组件传递

4、验证props

- 封装组件时，可以在子组件中对从父组件传递过来的props数据进行合法性校验，从而防止出现数据不合法的问题。

字符串数组形式的props

缺点

- 无法为每个prop指定具体的数据类型。

对象形式的props

优点

- 可对每个prop进行数据类型的校验。
- 可使用多种验证方案，包括基础类型检查、必填项的校验、属性默认值、自定义验证函数等。在声明props时，可以添加验证方案。

8.4.1 父组件向子组件传递

4、验证props

1. 基础类型检查

- 在开发中，有时需要对从父组件中传递过来的props数据进行基础类型检查，这时可以通过type属性检查合法的类型，如果从父组件中传递过来的值不符合此类型，则会报错。

- 常见的类型

- String (字符串)
- Number (数字)
- Boolean (布尔值)
- Array (数组)
- Object (对象)
- Date (日期)
- Function (函数)
- Symbol (符号)
- 任何自定义构造函数。

为props指定基础类型检查

```
props: {  
  自定义属性A: String,    // 字符串  
  自定义属性B: Number,    // 数字  
  自定义属性C: Boolean,   // 布尔值  
  自定义属性D: Array,     // 数组  
  自定义属性E: Object,    // 对象  
  自定义属性F: Date,      // 日期  
  自定义属性G: Function,  // 函数  
  自定义属性H: Symbol,    // 符号  
}
```


8.4.1 父组件向子组件传递

4、验证props

通过配置对象的形式定义验证规则，示例如下。

```
props: {  
  自定义属性: { type: Number },  
}
```

如果某个prop的类型不唯一，可以通过数组的形式为其指定多个可能的类型。

```
props: {  
  自定义属性: { type: [String, Array] }, // 字符串或数组  
}
```

8.4.1 父组件向子组件传递

4、验证props

2. 必填项的校验

父组件向子组件传递props数据时，有可能传递的数据为空，但是在子组件中要求该数据是必须传递的。可以在声明props时通过required属性设置必填项，强调组件的使用者必须传递属性的值，代码如下。

```
props: {  
  自定义属性: { required: true },  
}
```

3. 属性默认值

在声明props时，可以通过default属性定义属性默认值，当父组件没有向子组件的属性传递数据时，属性将会使用默认值，代码如下。

```
props: {  
  自定义属性: { default: 0 },  
}
```

8.4.1 父组件向子组件传递

4、验证props

4. 自定义验证函数

如果需要对从父组件中传入的数据进行验证，可以通过`validator()`函数来实现。`validator()`函数可以将`prop`的值作为唯一参数传入自定义验证函数，如果验证失败，则会在控制台中发出警告。为`prop`属性指定自定义验证函数的示例代码如下。

```
props: {  
  自定义属性: {  
    validator(value) {  
      return ['success', 'warning', 'danger'].indexOf(value) !== -1;  
    },  
  },  
}
```

»» 8.4.2 子组件向父组件传递

1、在子组件中声明自定义事件

若想使用自定义事件，首先需要在子组件中声明自定义事件。

非语法糖

emits选项声明自定义事件。

```
<script>
export default {
  emits: ['demo']
}
</script>
```

使用setup语法糖，需要通过调用
defineEmits() 函数声明自定义事件。

```
<script setup>
  const emit = defineEmits(['demo'])
</script>
```

defineEmits()函数的参数与emits选项中的相同。

8.4.2 子组件向父组件传递

2、在子组件中触发自定义事件

- 在子组件中声明自定义事件后，接着需要在子组件中触发自定义事件。
- 场景简单可以使用内联事件处理器，通过调用`$emit()`方法触发自定义事件，将数据传递给使用的组件，示例代码如下。

```
<button @click="$emit('demo', 1)">按钮</button>
```

说明：`$emit()`方法

- 第1个参数为字符串类型的自定义事件的名称，
- 第2个参数为需要传递的数据，当触发当前组件的事件时，该数据会传递给父组件。

8.4.2 子组件向父组件传递

2、在子组件中触发自定义事件

除了使用内联方式外，还可以直接定义方法来触发自定义事件。

从 `setup()` 函数的第 2 个参数（`setup` 上下文对象）来访问到 `emit()` 方法。

```
export default {  
  setup(props, ctx) {  
    const update = () => {  
      ctx.emit('demo', 2)  
    }  
    return { update }  
  }  
}
```

方法需要通过对象调用：
`object.emit(...)`;

使用 `setup` 语法糖，可以调用 `emit()` 函数来实现。

```
<script setup>  
  const update = () => {  
    emit('demo', 2)  
  }  
</script>
```

函数可以直接调用或通过参数传递：
`emit(...)`; // 直接调用

8.4.2 子组件向父组件传递

3、在父组件中监听自定义事件

- 在父组件中通过v-on可以监听子组件中抛出的事件，示例如下。

```
<子组件名 @demo="fun" />
```

说明：当触发demo事件时，会接收到从子组件中传递的参数，同时会执行fun()方法。
父组件可以通过value属性接收从子组件中传递来的参数。

- 在父组件中定义fun()方法，示例代码如下。

```
const fun = value => {  
  console.log(value)  
}
```

8.4.2 子组件向父组件传递

4、 在父组件中监听自定义事件示例

① 创建src\components\CustomSubComponent.vue文件——子组件。

```
1 <template>
2   <p>count值为: {{ count }}</p>
3   <button @click="add">加n</button>
4 </template>
5 <script setup>
6   import { ref } from 'vue'
7   const emit = defineEmits(['updateCount']) 声明自定义事件updateCount
8   const count = ref(1)
9   const add = () => { count.value++
10    |   emit('updateCount', 2) } 直接定义方法触发自定义事件
11 </script>
```

② 创建同一目录下
CustomEvents.
vue文件——父
组件

```
1 <template>
2   <p>父组件当前的值为: {{ number }}</p>
3   <CustomSubComponent @updateCount="updateEmitCount" />
4   </template>
5 <script setup>
6   import CustomSubComponent from './CustomSubComponent.vue'
7   import { ref } from 'vue'
8   const number = ref(10)
9   const updateEmitCount = (value) => { number.value += value }
10 </script>
```

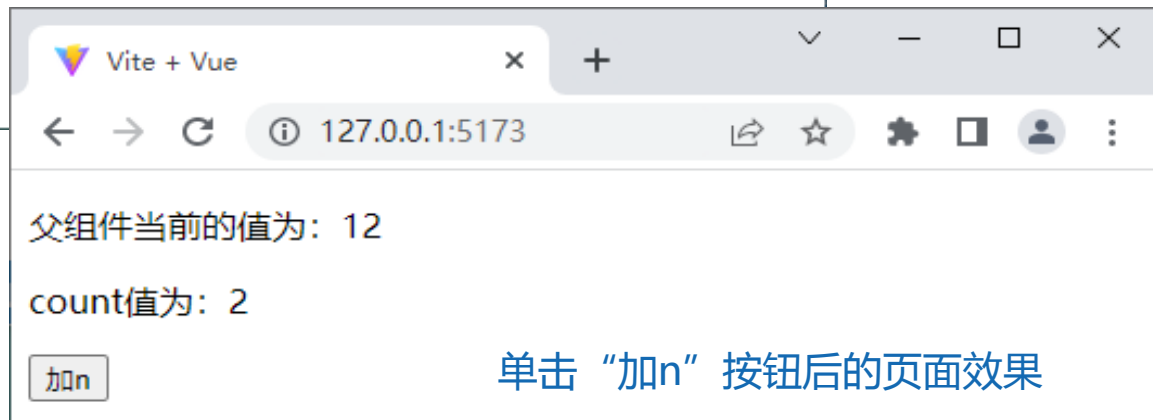
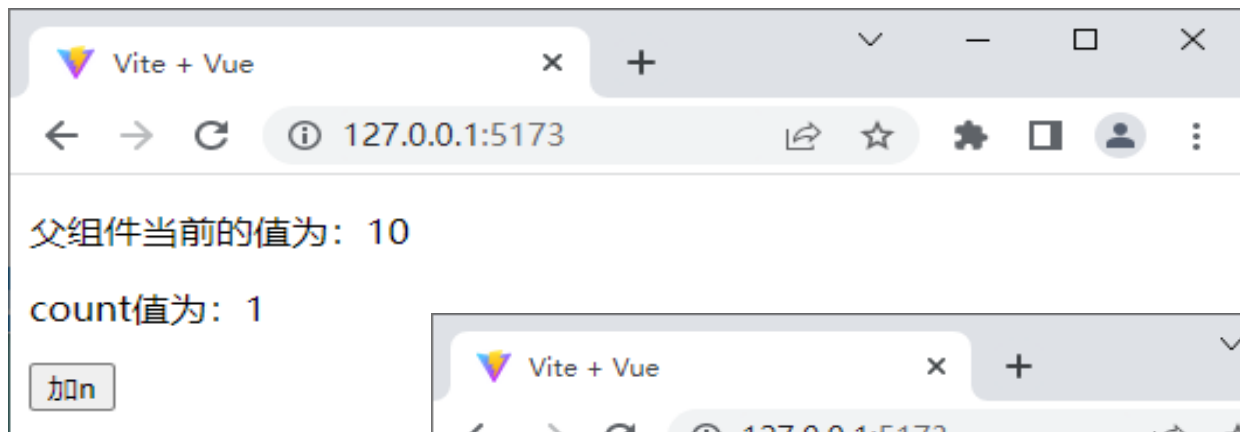

8.4.2 子组件向父组件传递

4、在父组件中监听自定义事件

③修改src\main.js文件，切换页面中显示的组件。

```
import App from './components/CustomEvents.vue'
```

④在浏览器中访问http://127.0.0.1:5173/，初始页面效果如下图所示。



单击“加n”按钮后的页面效果

8.4.3 跨级组件之间的数据传递

- Vue提供了跨级组件之间数据传递的方式——依赖注入。
- 父组件是其所有的后代组件的依赖提供者。而任何后代的组件树，无论层级多深，都可以注入由父组件提供的依赖。
- 父组件为后代组件提供数据，需要使用`provide()`函数。
- 子组件想要注入上层组件或整个应用提供的数据，需要使用`inject()`函数。

8.4.3 跨级组件之间的数据传递

1. provide()函数

- provide()函数可以提供一個值，用于被后代组件注入。
- provide()函数的语法格式如下。

```
provide(注入名, 注入值)
```

- provide()函数2个参数：
 - 第1个注入名，后代组件会通过注入名查找所需的注入值；
 - 第2个注入值，值可以是任意类型，包括响应式数据。
(例如通过ref()函数创建的数据)

8.4.3 跨级组件之间的数据传递

provide()函数必须在组件的setup()函数中调用。

```
<script>
import { ref, provide } from 'vue'
export default {
  setup() {
    const count = ref(1)
    provide( 'message', count )
  }
}
</script>
```

使用setup语法糖，provide()函数。

```
<script setup>
import { provide } from 'vue'
provide('message', 'Hello Vue.js')
</script>
```

provide()函数除了可以在某个组件中提供依赖外，还可以**全局提供依赖**。例如，在src\main.js文件中添加全局依赖。

```
const app = createApp(App)
app.provide('message', 'Hello Vue.js')
```

8.4.3 跨级组件之间的数据传递

2. inject()函数

- 通过inject()函数可以注入上层组件或者整个应用提供的数据。
- inject()函数的语法格式如下。

```
inject(注入值, 默认值, 布尔值)
```

- inject()函数3个参数：

- 第1个参数是注入值，Vue会遍历父组件，通过匹配注入的值来确定所提供的值。如果父组件链上多个组件为同一个数据提供了值，距离更近的会覆盖更远的组件所提供的值。
- 第2个参数（可选），用于在没有匹配到注入的值时使用默认值。
 - 工厂函数：用于返回某些创建起来比较复杂的值。
 - 函数：还需要将false作为第3个参数传入，表明这个函数就是默认值。
- 第3个参数（可选），类型为布尔值。
 - False：表示默认值是函数；
 - True：表示默认值为工厂函数；
 - 省略参数值：表示默认值为其他类型的数据，不是函数或工厂函数。

8.4.3 跨级组件之间的数据传递

inject()函数必须在组件的setup()函数中调用。

```
1  <script>
2  import { inject } from 'vue';
3  export default {
4    setup() {
5      const count = inject('count')
6      const foo = inject('foo', 'default value')
7      const baz = inject('foo', () => new Map())
8      const fn = inject('function', () => { }, false)
9    }
10 }
11 </script>
```

注入一个值，若为空
则使用提供的默认值

没有匹配到注入值时，
默认值可以是工厂函数

为表明提供的默认值是函
数，需要传入第三个参数

使用setup语法糖，inject()函数代码。

```
1  <script setup>
2  import { inject } from 'vue';
3  const count = inject('count')
4  </script>
```

8.4.3 跨级组件之间的数据传递

※跨级组件之间的数据传递示例※

① ProvideChildren.vue

文件——子组件。

```
<template>
  <div>子组件</div>
</template>
```

② ProvideParent.vue文件——父组件

```
1  <template>
2    <div>父组件</div>
3    <hr>
4    <ProvideChildren />
5  </template>
6  <script setup>
7    import ProvideChildren from './ProvideChildren.vue'
8  </script>
```

③ ProvideGrand.vue文件——爷爷组件

```
1  <template>
2    <div>爷爷组件</div> <hr>
3    <ProvideParent />
4  </template>
5  <script setup>
6    import ProvideParent from './ProvideParent.vue'
7    import { ref, provide } from 'vue'
8    let money = ref(1000)
9    let updateMoney = (value) => { money.value += value }
10   provide('money', money)
11   provide('updateMoney', updateMoney)
12 </script>
```

8.4.3 跨级组件之间的数据传递

※跨级组件之间的数据传递示例※

④修改ProvideChildren.vue文件，通过inject()函数接收爷爷组件中传过来的数据。

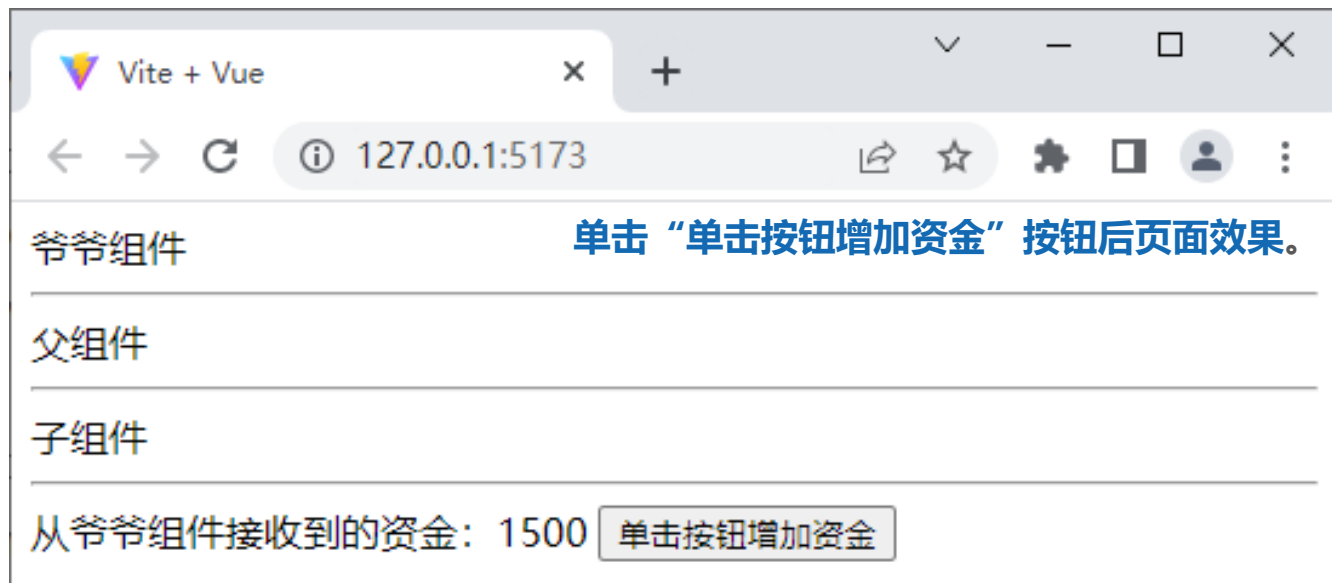
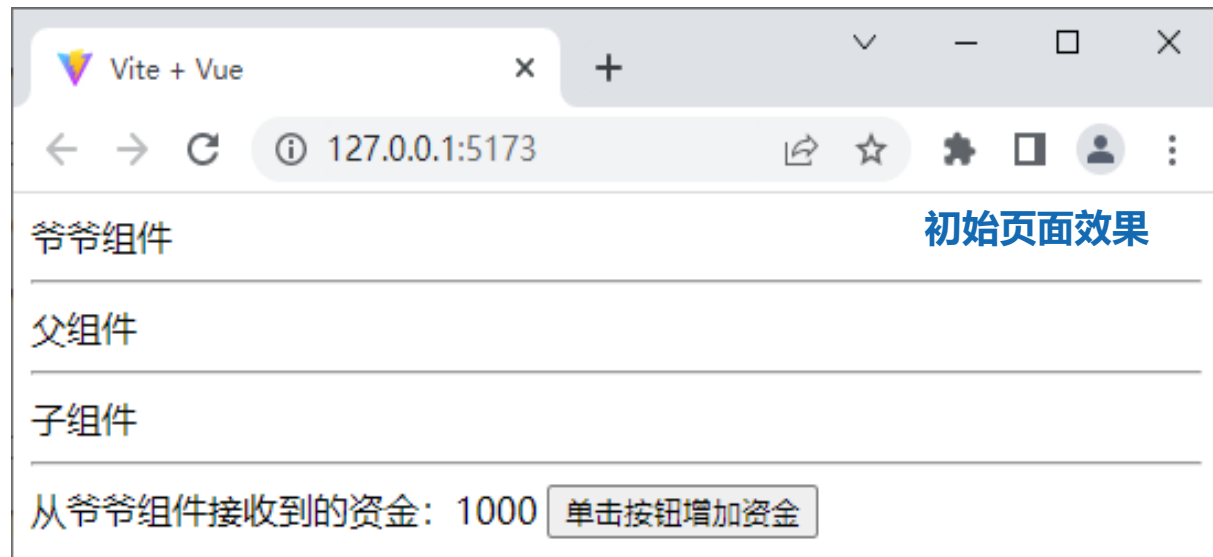
```
1  <template>
2    <div>子组件</div> <hr>
3    从爷爷组件接收到的资金: {{ money }}
4    <button @click="updateMoney(500)">单击按钮增加资金</button>
5  </template>
6  <script setup>
7    import { inject } from 'vue'
8    let money = inject('money')
9    let updateMoney = inject('updateMoney')
10  </script>
```

⑤修改src\main.js文件，切换页面中显示的组件。

```
import App from './components/ProvideGrand.vue'
```


>>> 8.4.3 跨级组件之间的数据传递

※跨级组件之间的数据传递示例※



```
<template>
  <div class="app">
    <h3>我是App组件（祖），{{name}}--{{price}}</h3>
    <Child/>
  </div>
</template>

<script>
import { reactive,tRefs,provide } from 'vue'
import Child from './components/Child.vue'
export default {
  name:'App',
  components:{Child},
  setup(){
    let car = reactive({name:'奔驰',price:'40W'})
    provide('car',car) //给自己的后代组件传递数据
    return {...tRefs(car)}
  }
}
</script>

<style>
.app{
  background-color: gray;
  padding: 10px;
}
</style>
```

```
<template>
  <div class="child">
    <h3>我是Child组件（子）</h3>
    <Son/>
  </div>
</template>

<script>
import Son from './Son.vue'
export default {
  name:'Child',
  components:{Son}
}
</script>

<style>
.child{
  background-color: skyblue;
  padding: 10px;
}
</style>
```



```
<template>
  <div class="son">
    <h3>我是Son组件（孙）， {{car.name}}--{{car.price}}</h3>
  </div>
</template>

<script>
  import {inject} from 'vue'
  export default {
    name: 'Son',
    setup(){
      let car = inject('car')
      return {car}
    }
  }
</script>

<style>
  .son{
    background-color: orange;
    padding: 10px;
  }
</style>
```

我是App组件（祖）， 奔驰--40W

我是Child组件（子）

我是Son组件（孙）， 奔驰--40W

本章小结

Vue组件的基础知识：

- 选项式API和组合式API
- 生命周期函数
- 组件的注册和引用
- 组件之间的样式冲突
- 组件之间的数据传递
 - 父 ↔ 子
 - 跨级